

2.1 PPT

2.1 Function Calling 与 Tool Use：让模型提出动作，让 Runtime 安全执行

一、导论

1.1 本章重点

让 Agent 不仅能说，还能做

- 第一章：让模型调用统一、稳定、可观察
- 第二章：让模型提出的动作进入统一、受控、可追踪的运行时
- 核心目标：让 Agent 做得安全、规范、可观测

第一章留下的问题：Agent 能回答问题，但还不能完成任务

- “帮我查订单”——只能告诉用户应该怎样查询
- “取消这笔订单”——没有真实数据、系统连接和业务权限
- 模型有“大脑”，但没有“手”和“工具”

第二章要解决什么？

1. 给 Agent 装上“手”和“工具”

- Function Calling：模型结构化地提出动作
- Tool Runtime：校验并安全执行动作
- MCP：标准化接入外部工具和资源

2. Function Calling 的核心边界：模型负责提议，Runtime 负责执行

角色	负责什么
模型	理解目标、选择工具、生成参数
Agent Loop	推进模型调用、工具调用和结果回写
Tool Runtime	校验、鉴权、执行、审计、追踪
业务 Handler	完成真实业务操作

Tool Call 是候选动作，不是执行许可。

3. 一次工具调用的主链：从用户目标到业务结果

- a. 模型读取工具名称、描述和 Input Schema
- b. 模型生成 Tool Call
- c. Runtime 校验参数、权限和风险
- d. Handler 执行真实业务操作
- e. Runtime 生成 Tool Result
- f. Agent Loop 携带 `tool_call_id` 写回模型
- g. 模型继续调用工具或返回最终答案

4. Tool Runtime 解决的问题：让 Tool Call 进入受控执行

- 工具注册：工具怎样定义和管理
- 工具发现：当前任务应该看到哪些工具
- 快照冻结：模型和 Runtime 使用同一版本
- 分阶段执行：`prepare → execute → finalize`
- 失败控制：超时、重试、降级、结构化错误
- 可追踪性：Audit、Trace、`tool_call_id`

5. 工具治理与安全边界：模型可以提出动作，但不能决定权限

- `user_id`、角色、租户、审批状态：来自可信上下文
- 只读、写操作、高风险操作：使用不同执行路径
- 参数错误、越权、未确认：必须在 Handler 之前拒绝
- 非幂等写操作：不能因为超时就盲目重试
- 验收重点：非法调用不会产生真实副作用

6. MCP 解决什么问题？

从“单个工具接入”走向“标准化连接外部能力”

- MCP Host：承载 Agent 和运行策略
- MCP Client：连接外部 Server
- MCP Server：暴露 Tool、Resource、Prompt

- 支持文件系统、数据库、GitHub、内部 API 等能力

MCP 与 Function Calling 的区别

Function Calling	MCP
模型如何选择工具并生成参数	外部能力如何被发现和标准化接入
面向一次模型调用	面向工具、资源和 Prompt 的连接协议
仍需 Runtime 负责执行	仍需本地 Policy 和 Runtime 治理

7. Agent Tool Runtime：能力装配：把分散知识点装配成统一平台

- Tool Registry
- Function Calling
- JSON Schema / Pydantic 校验
- 权限控制与风险分级
- 超时、重试、降级与失败恢复
- MCP Tool 接入
- Audit、Trace 与 CLI 确定性验收

8. 阶段二项目：Agent 工具调用基础设施

在阶段一 LLM Gateway 之上增加行动基础设施

- 配置文件：接入 DeepSeek OpenAI-compatible API
- CLI：绕开模型，确定性验收 Runtime
- DeepSeek：完成真实 Function Calling 集成测试
- Web：展示工具、用量、审计和 Trace
- 测试：证明非法调用不会进入 Handler
- 统一原则：CLI、模型、MCP 共用同一个 Tool Runtime

工程师必须亲自完成的工作

AI 可以补代码，但不能替工程师做边界判断

- 判断什么事情值得做成 Tool
- 设计工具粒度、输入输出和错误协议
- 区分模型字段与可信字段

- 决定权限、风险、审批和重试规则
- 设计故障注入、回归测试和验收证据
- 审查 AI 生成代码是否绕过 Runtime 或丢失调用关联

学完本章的价值

从“会调用模型”走向“能构建 Agent 基础设施”

- 能让 Agent 读取真实数据、调用外部能力并完成业务任务
- 能把不确定的模型输出收进确定性的执行边界
- 能读懂 DeepSeek Harness、`pi`、Codex 等框架的工具与 Loop 设计
- 能发现常见工程问题：越权、盲目重试、结果错配、治理旁路
- 能为后续 Agent Loop、状态机、Memory 和生产化部署打基础

1.2 当前还需要古法手写整套 Function Calling 代码吗？

先说结论：**不需要**。

本节真正训练的是四项能力。

1. 构建与验证工具调用闭环的能力

能把 Tool Schema、Tool Call、Runtime 执行、Tool Result 和模型下一轮决策串成完整链路；还能用测试证明参数非法、权限不足或缺少确认时，handler 没有执行。

2. 软件工程判断能力

能区分模型协议、业务函数、Tool Runtime 和 Agent Loop 的职责；能识别不可信输入、可信上下文、副作用边界和失败语义，而不是把所有逻辑都塞进一个函数。

3. 与 AI 协作开发的能力

能先定义工具边界、Schema 和验收条件，再让 AI 补实现；拿到 AI 代码后，能审查是否绕过校验、相信模型提供的身份、丢失 tool_call_id，或者对非幂等写操作盲目重试。

4. 迁移与应用能力

能把同一套方法从订单查询迁移到工单、知识库、文件系统或企业 API，而不是只会复现课堂里的 search_orders。

1.3 本节需要掌握哪些技能？

你应当能够做到三件事：

- 不看代码，用两分钟讲清“模型提出动作，Runtime 受控执行，Loop 回写结果”的主链；
- 面对一段 AI 生成的工具调用代码，指出协议、权限、重试和结果关联上的关键问题；
- 为一个新业务动作写出最小工具定义，能构造 Tool Call，并生成与调用 ID 正确配对的 Tool Result。

1.4 本节围绕哪条主线展开？

本节围绕 Function Calling 的六个基础问题：

小节	本节要回答的问题
1.1	模型怎样提出 Tool Call，结果又怎样回到模型？
1.2	一个业务函数怎样定义成模型可选择、Runtime 可治理的工具？
1.3	Input、Output 和 Error Schema 分别保护哪一段边界？
1.4	模型依据什么选择工具，最小 Agent Loop 怎样继续下一轮？
1.5	为什么 Tool Call 不能直接执行，治理边界至少要检查什么？
1.6	为什么工具能力要与 Agent Loop 解耦，插件化需要保留哪些接口？

六个问题最终收束到一条主线：

代码块

```
1  模型提出 Tool Call
2    → Runtime 校验、授权并执行
3    → 生成带原始 tool_call_id 的 Tool Result
4    → Agent Loop 写回结果
5    → 模型继续决策或完成回答
```

在这条链路里，模型擅长理解意图、选择候选工具和生成候选参数；工程师负责定义协议、可信边界、权限风险、失败恢复和验收标准。

二、先划清边界：模型已经能回答问题，为什么还需要 Tool Use？

比如用户问：“帮我查一下昨天创建、还没有支付的订单。”模型可以解释应该怎样查询，但它不知道业务数据库里的实时订单，也不能只凭一句自然语言直接访问数据库。

如果用户进一步说：“把这笔订单取消掉”，问题就更明显了。模型不仅缺少实时数据，也不应该拥有数据库连接、用户身份和取消订单的真实权限。

所以**Function Calling** 是让模型用结构化方式提出“我想调用哪个工具、参数是什么”。真正的执行必须交给 **Tool Runtime**。

一条可靠的职责边界是：

角色	负责什么	不能替谁决定什么
模型	理解请求、选择候选工具、生成候选参数、阅读工具结果	不能决定自己是否有权限，也不能直接执行 Python 函数
工具函数	完成一项确定的业务能力	不能相信调用参数天然正确，也不负责整个 Agent Loop
Tool Runtime	查找工具、校验、鉴权、超时、重试、审计和结果归一化	不能替业务方定义权限和风险规则
Agent Loop / Harness	保存消息和状态，把 Tool Result 送回模型，决定继续或结束	不应绕过 Runtime 直接调用业务函数

下面代码分散在三个源码文件中，这里省略异常与分支处理；文件注释标明了对应定义在 pi 中的位置。

pi_types.ts

三个重点函数：

- 第一，Tool 没有 execute()。
- 第二，prepareToolCall() 先调用 validateToolArguments()，再运行 beforeToolCall。
- 第三，executePreparedToolCall()、finalizeToolCall() 和 createToolResultMessage() 始终沿用同一个调用 ID。

接下来，我们先把这条链路拆开，看一次 Function Calling 到底发生了什么。

1.1 Function Calling 工作机制

为什么模型要返回 Tool Call，而不直接执行？

先看结论：模型返回的 Tool Call 只是一个候选动作，它与用户输入一样，都属于不可信数据。

一次完整调用通常包含六步：

1. 应用把用户消息和当前可用的工具 Schema 一起发送给模型；
2. 模型决定直接回答，或者返回一个或多个 Tool Call；
3. 每个 Tool Call 携带调用 ID、工具名称和 JSON 参数；
4. Tool Runtime 校验工具、参数、权限和风险，再执行真实函数；
5. Runtime 把成功结果或结构化错误作为 Tool Result 写回消息历史；
6. 模型读取结果，继续调用工具、向用户追问，或者生成最终答案。

这条链路可以压缩成：

代码块

```
1  User Message
2      ↓
3  LLM: 直接回答，或生成 Tool Call
4      ↓
5  Tool Runtime: validate → authorize → execute → observe
6      ↓
7  Tool Result: 必须带回原来的 tool_call_id
8      ↓
9  LLM: 继续决策或完成回答
```

这里最容易被忽略的是 tool_call_id。tool_call_id 是一次“模型提出的工具调用”的关联键。它把 Assistant Message 中的 Tool Call、Runtime 中的执行过程，以及随后写回的 Tool Result 串成同一条记录。

假设用户说：“查询 ord_123 和 ord_456 的状态。”模型可能在同一条 Assistant Message 中提出两个调用：

代码块

```
1  {
2    "role": "assistant",
3    "tool_calls": [
```

```

4      {
5          "id": "call_A",
6          "function": {
7              "name": "get_order",
8              "arguments": "{\"order_id\":\"ord_123\"}"
9          }
10     },
11     {
12         "id": "call_B",
13         "function": {
14             "name": "get_order",
15             "arguments": "{\"order_id\":\"ord_456\"}"
16         }
17     }
18 ]
19 }

```

两个调用的工具名称完全相同，区别只在参数和调用 ID。

如果 Runtime 只记录工具名，就无法稳定区分两次执行。

而且并行执行时，完成顺序不一定等于生成顺序。

Runtime 可以先后得到下面两份结果：

代码块

```

1  {
2      "role": "tool",
3      "tool_call_id": "call_B",
4      "content": "{\"order_id\":\"ord_456\",\"status\":\"paid\"}"
5  }

```

代码块

```

1  {
2      "role": "tool",
3      "tool_call_id": "call_A",
4      "content": "{\"error\":{\"code\":\"TIMEOUT\"}}"
5  }

```


虽然结果顺序发生了变化，模型仍然可以通过 call_B 知道 paid 属于 ord_456，通过 call_A 知道 ord_123 查询超时。

所以，tool_call_id 至少解决四个问题。

第一，解决消息协议的对应关系。

第二，解除生成顺序与完成顺序的耦合。

第三，区分成功和失败的恢复对象。

第四，建立 Trace 的父子关系。

Runtime 在并行执行时，应当让调用 ID 跟随调用对象进入每个异步任务，而不是等工具完成后再按返回顺序猜测：

tool_batch.py

代码块

```
1  import asyncio
2
3
4  async def execute_bound(
5      call: ToolCall,
6      runtime: "ToolRuntime",
7      ctx: "ExecutionContext",
8  ) -> tuple[str, ToolResultMessage]:
9      result = await runtime.execute(call, ctx)
10     return call.id, result
11
12
13  async def execute_batch(
14      calls: list[ToolCall],
15      runtime: "ToolRuntime",
16      ctx: "ExecutionContext",
17  ) -> list[ToolResultMessage]:
18      tasks = [
19          asyncio.create_task(execute_bound(call, runtime, ctx))
20          for call in calls
21      ]
22
23      results: list[ToolResultMessage] = []
24      for task in asyncio.as_completed(tasks):
25          call_id, result = await task
26          assert result.tool_call_id == call_id
27          results.append(result)
```

```
28
29     return results
```

这段代码演示 Harness 怎样安全执行一批可以并行的 Tool Call。execute_batch() 为每个调用创建独立任务，execute_bound() 则把调用 ID 与 Runtime 返回结果绑定在同一个协程返回值中。

还要注意，Tool Result 不等于最终答案，它只是新的 observation。

代码块

```
1  模型决策 → 工具执行 → 获得观察 → 模型再次决策
```

本小节结论

Function Calling 只负责把模型的动作意图变成结构化 Tool Call；Tool Runtime 才负责把候选动作变成受控执行；Agent Loop 再把 Tool Result 变成下一轮上下文。三者不能合并成一次普通函数调用。

1.2 工具定义：名称、描述、Schema、权限与风险

为什么不能把一个普通 Python 函数直接暴露给模型？

Agent 中的工具不是一个裸函数，而是一份“模型可理解、Runtime 可执行、平台可治理”的业务协议。

每个工具至少要有以下信息：

字段	解决的问题	示例
name	稳定、唯一地标识工具	search_orders

description	告诉模型何时使用、边界是什么	“查询当前用户的订单，不修改订单”
input_model	调用需要哪些参数	状态、开始日期、数量上限
output_model	成功后返回什么事实	订单列表、总数
error_model	失败后怎样分类和恢复	参数错误、拒绝、超时、上游失败
permission	调用者需要什么权限	order:read
risk	调用前需要怎样的控制	low、medium、high

下面用 Python 定义统一的工具元数据。它参考 pi 中“Tool 元数据与 AgentTool 执行函数分离”的思路，同时增加课程当前需要的输出、错误、权限和风险字段。

tool_definition.py

这段代码定义工具的最小完整契约：模型选择需要的名称、描述和输入 Schema；应用侧需要的输出、错误、权限和风险；真正完成业务操作的 handler。超时、重试、版本和依赖等运行策略会在 2.2 的 Tool Runtime 中继续补充。

工具名称和描述需要工程师亲自设计，不能让模型随意生成。一个可用的描述至少要回答三件事：

代码块

```
1  它做什么？
2  什么时候应该使用？
3  它不会做什么？
```

例如：

代码块

```
1  名称：search_orders
2  描述：按当前用户、订单状态和创建日期查询订单；只读取订单，不能取消、退款或修改订单。
```

而下面这种描述几乎没有选择价值：

代码块

```
1  名称：query_data
2  描述：查询数据。
```

本小节结论

工具定义不是函数注释，而是跨越模型、Runtime 和业务系统的执行协议。

工程师必须确定工具粒度、用途边界、数据结构、权限、风险和失败策略；模型只能在这份协议内提出调用。

模型可以生成字段，却不能替业务系统确定必填条件、跨字段关系、稳定返回值和错误恢复语义。所以下一步不是继续增加工具描述，而是把输入、输出和错误写成真正可以校验的类型。

1.3 Input Schema、Output Schema 与 Error Schema

为什么只校验输入还不够？

输入校验只能阻止模型把错误参数送进业务函数，却不能保证业务函数返回了可用结果，也不能告诉 Agent 失败后应该怎样继续。

因此要把工具边界拆成三份 Schema：

Schema	保护哪一侧	核心问题
Input Schema	模型到 Runtime	参数是否完整、类型是否正确、范围是否合理
Output Schema	业务系统到 Agent	成功结果是否稳定、后续 Loop 能否直接消费
Error Schema	失败到恢复逻辑	失败属于哪一类、能否重试、下一步是什么

我们继续参考 pi 源码，使用订单查询工具：

order_schemas.py

这段代码一次定义了工具边界上的三份可执行协议。

SearchOrdersInput 验收模型参数，SearchOrdersOutput 验收业务函数返回值，ToolError 统一表达失败。

三者都继承 StrictModel，从默认行为上拒绝额外字段和隐式类型转换。

Input Schema 重点看三层约束。

Output Schema 重点看它返回的是订单 ID、状态、时间和金额。

Error Schema 重点看 code 与 retryable 的分工。例如：

- INVALID_ARGUMENT：模型可以修正参数后再次调用；
- PERMISSION_DENIED：不能重试同一调用，应停止或提示用户切换身份；
- APPROVAL_REQUIRED：需要进入人工确认流程；
- TIMEOUT、UPSTREAM_ERROR：只有满足幂等条件时，Runtime 才可以有限重试；
- INVALID_OUTPUT：说明工具实现违反了自己的返回协议，应记录工程故障，而不是让模型猜测缺失字段。

把这段代码放回 Loop，可以看到三份 Schema 分别卡在不同位置：

- Input 位于 Tool Call 与 handler 之间；
- Output 位于 handler 与 Tool Result 之间；
- Error 位于失败与下一轮决策之间。

现在把 Schema 与真实工具函数装配起来：

search_orders_tool.py

这段代码完成两件事：

1. search_orders() 实现真实业务查询；SEARCH_ORDERS 再把 handler 与前面定义的 Schema、权限和风险装配成 Agent Tool。单独存在的异步函数还不是模型可以使用的工具，应用还需要为它提供稳定协议。
2. permission 与 risk 只声明治理要求，具体怎样执行由下一节的 Tool Runtime 负责。

本小节结论

Input Schema 拦截错误调用，Output Schema 固定成功事实，Error Schema 驱动失败恢复。

三者共同构成工具协议；缺少任何一份，Agent Loop 都会被迫解析自然语言或猜测下一步。

1.4 工具选择：模型怎样选择工具并生成参数？

模型是怎样知道该调用哪个工具的？

工具选择质量主要受四个因素影响：

因素	工程师需要控制什么
工具集合	当前请求真正允许模型看到哪些工具
名称与描述	每个工具的用途、适用条件和禁止边界
参数 Schema	字段含义、枚举、必填项和范围
上下文	用户目标、已获得的 Tool Result 和当前状态

不要把所有平台工具都暴露给每一轮模型。

下面使用 DeepSeek V4 Flash 和 OpenAI Compatible Chat Completions 演示一次工具选择。

model_select.py

1. tools=[SEARCH_ORDERS.to_model_tool()] 证明发送给模型的是公开 Schema，而不是 Python 函数；
2. tool_choice="auto" 允许模型根据任务选择直接回答或调用工具；
3. assistant.tool_calls 可能为空、一个或多个，所以 Harness 不能假设每轮必定执行工具；
4. call.function.arguments 仍然是模型生成的 JSON 字符串，打印出来只能用于观察，不能直接展开调用 handler。

把模型决策、工具结果和下一次模型调用串起来，可以得到最小 Tool Loop：

minimal_tool_loop.py

这段代码第一次把模型调用和 Tool Runtime 组成最小 Agent Loop。它接收用户目标，持续执行“请求模型—检查 Tool Call—执行工具—写回结果”，直到模型不再调用工具并返回最终文本。

本小节结论

模型根据“当前上下文 + 当前可见工具 Schema”选择候选工具并生成参数。

工具描述影响选择，Schema 影响参数，但两者都不能替代 Runtime 校验和授权。

Tool Call 是提议，不是执行许可。

1.5 工具治理：本节需要先建立哪些执行边界？

为什么模型已经选对工具，程序仍然不能直接执行？

最小执行顺序是：

代码块

- 1 接收 Tool Call
- 2 → 校验工具名称和 Input Schema
- 3 → 从可信上下文读取身份、权限和审批
- 4 → 判断风险、超时与重试边界
- 5 → 执行 handler
- 6 → 校验 Output 或构造 Error
- 7 → 生成带原始 tool_call_id 的 Tool Result
- 8 → 写入 Trace，再交回 Agent Loop

1.5.1 哪些信息来自模型，哪些信息必须由 Harness 注入？

模型只应该填写完成业务动作所需的参数。例如查询订单时，它可以填写状态和日期；不能填写“我是管理员”“本次已经审批”或者“把风险改成 low”。

可以把两类数据明确分开：

数据来源	典型字段	是否可信
Tool Call	订单号、查询条件、用户描述	不可信，必须经过 Input Schema 和业务规则校验
Execution Context	用户、租户、权限、审批记录、Trace ID	由登录态或服务间身份生成，Runtime 可以据此判断
Tool Definition	权限要求、风险等级、超时、是否允许重试	由工程师配置，模型不能修改
handler 结果	数据库或外部 API 返回的数据	仍需经过 Output Schema，不能直接进入模型

1.5.2 超时以后，为什么不能统一重试？

因为超时只表示调用方没有及时拿到结果，不代表业务动作一定失败。

场景	本节先记住的处理边界
查询订单遇到连接抖动	只读且参数不变，可以在预算内有限重试
取消订单请求超时	可能已经取消，不能直接再执行一次
参数校验失败	不应由 Runtime 原参数重试，应把安全错误返回 Agent Loop
权限或审批不足	必须停止，不能靠重试绕过
handler 返回结构错误	转成稳定错误，不能把原始对象直接交给模型

务必区分两种“再试一次”：

代码块

1

Runtime 重试：工具和参数不变，只恢复明确的瞬时故障

2

Agent Loop 修正：模型阅读 Tool Result 后，重新选工具、改参数或询问用户

本小节结论

工具调用的第一步先建立执行不变量：模型参数不可信，身份权限来自 Harness，拒绝必须发生在 handler 之前，写操作不能因超时盲目重试，所有结果必须保留原始 tool_call_id。

本节最后只再看一个更高层的问题：为什么工具能力不应该直接写进 Agent Loop？

三、本节小结：Function Calling 最终解决了什么问题？

Function Calling 把模型的自然语言决策变成结构化候选动作。

建立了下面这条协议闭环：

代码块

- 1 工程师定义 Tool
- 2 → 模型看到名称、描述和 Input Schema
- 3 → 模型生成带 ID 的 Tool Call
- 4 → 应用校验并调用工具
- 5 → 应用生成带原始 tool_call_id 的 Tool Result
- 6 → Agent Loop 把结果交给模型继续决策

本节你需要知道：

1. Tool Call 是不可信的动作提议，不是执行许可；
2. 名称和描述影响模型是否选对工具，Input Schema 约束模型可以填写什么；
3. Output Schema 和 Error Schema 让结果成为稳定协议，而不是随意字典；
4. 模型可见 Tool 与 Runtime 可执行 AgentTool 必须分离，handler 不会暴露给模型；
5. 无论成功还是失败，Tool Result 都必须保留原始 tool_call_id。

工程师负责工具粒度、字段语义、可信数据来源、风险等级和结果协议；

AI 可以生成 Schema、数据类和适配代码，但不能替业务系统决定这些边界。

四、课堂练习

4.1 两分钟讲清协议闭环？

不看代码，回答下面五个问题：

1. Function Calling 与普通函数调用有什么区别？
2. 模型侧 Tool 为什么没有 execute()？
3. Input、Output 和 Error Schema 分别约束什么？
4. 为什么权限和审批不能作为模型参数？
5. 两个同名工具调用乱序完成时，tool_call_id 怎样防止结果错配？

4.2 代码：能不能完成一次最小调用？

模仿瑞幸协议接口，<https://open.lkcoffee.com/docs>，为 search_orders 完成下面四步：

1. 定义名称、描述和 Input Schema；
2. 生成模型可见的 Tool JSON，确认其中没有 handler、权限和内部依赖；
3. 构造两条不同 tool_call_id 的 Tool Call；
4. 分别生成 Tool Result，并断言结果 ID 与原调用一致。

练习只需要使用模拟订单数据，不要求实现 Registry、Snapshot、完整权限系统或重试框架。

4.3 审查验收：能不能发现协议错误？

检查一段 AI 生成的 Function Calling 代码，至少识别下面四类问题：

- 把 user_id、role 或 approved 放入 Input Schema，让模型自己填写；
- 把完整 ToolDefinition 或 handler 信息发送给模型；
- 直接相信参数 JSON，没有经过 Pydantic 校验；
- Tool Result 只返回内容，没有原始 tool_call_id。

发现以后，要说明错误会影响模型选择、可信边界还是结果关联，并给出相应测试。